

Author: JF Bastien
 Contact: jfb@google.com
 Author: Olivier Giroux
 Contact: ogiroux@nvidia.com
 Date: 2016-03-03
 Previous: <http://wg21.link/N4523>
 Previous: <http://wg21.link/P0154R0>
 URL: <https://github.com/jfbastien/papers/blob/master/source/P0154R1.rst>

Rationale

Starting with C++11, the library includes `std::thread::hardware_concurrency()` to provide an implementation quantity useful in the design of control structures in multi-threaded programs: the extent of threads that do not interfere (to the first-order). Established practice throughout the industry also relies on a second implementation quantity, used instead in the design of data structures in the same programs. This quantity is the granularity of memory that does not interfere (to the first-order), commonly referred to as the cache-line size.

Uses of cache-line size fall into two broad categories:

- Avoiding destructive interference (false-sharing) between objects with temporally disjoint runtime access patterns from different threads. e.g. Producer-consumer queues.
- Promoting constructive interference (true-sharing) between objects which have temporally local runtime access patterns. e.g. The `barrier` example, as illustrated in [P0153R0](#).

The most significant issue with this useful implementation quantity is the questionable portability of the methods used in current practice to determine its value, despite their pervasiveness and popularity as a group. In the [appendix](#) we review several different compile-time and run-time methods. The portability problem with most of these methods is that they expose a micro-architectural detail without accounting for the intent of the implementors (such as we are) over the life of the ISA or ABI.

We aim to contribute a modest invention for this cause, abstractions for this quantity that can be conservatively defined for given purposes by implementations:

- Destructive interference size: a number that's suitable as an offset between two objects to likely avoid false-sharing due to different runtime access patterns from different threads.
- Constructive interference size: a number that's suitable as a limit on two objects' combined memory footprint size and base alignment to likely promote true-sharing between them.

In both cases these values are provided on a quality of implementation basis, purely as hints that are likely to improve performance. These are ideal portable values to use with the `alignas()` keyword, for which there currently exists nearly no standard-supported portable uses.

Proposed addition

Below, substitute the $\blacklozenge$ character with a number the editor finds appropriate for the sub-section. We propose adding the following to the standard:

Under 18.6 Header `<new>` synopsis [support.dynamic]:

```
namespace std {
  // ...
  // 18.6. $\blacklozenge$  Hardware interference size
  static constexpr size_t hardware_destructive_interference_size = implementation-defined;
  static constexpr size_t hardware_constructive_interference_size = implementation-defined;
  // ...
}
```

Under 18.6. $\blacklozenge$ Hardware interference size [hardware.interference]:

```
constexpr size_t hardware_destructive_interference_size = implementation-defined;
```

This number is the minimum recommended offset between two concurrently-accessed objects to avoid additional performance degradation due to contention introduced by the implementation. It shall be at least `alignof(max_align_t)`.

[Example:

```
struct keep_apart {
  alignas(hardware_destructive_interference_size) atomic<int> cat;
  alignas(hardware_destructive_interference_size) atomic<int> dog;
};
```

— end example]

```
constexpr size_t hardware_constructive_interference_size = implementation-defined;
```

This number is the maximum recommended size of contiguous memory occupied by two objects accessed with temporal locality by concurrent threads. It shall be at least `alignof(max_align_t)`.

[Example:

```
struct together {
    atomic<int> dog;
    int puppy;
};
struct kennel {
    // Other data members...
    alignas(sizeof(together)) together pack;
    // Other data members...
};
static_assert(sizeof(together) <= hardware_constructive_interference_size);
```

– end example]

The `__cpp_lib_thread_hardware_interference_size` feature test macro should be added.

Appendix

Compile-time cache-line size

We informatively list a few ways in which the L1 cache-line size is obtained in different open-source projects at compile-time.

The Linux kernel defines the `__cacheline_aligned` macro which is configured for each architecture through `L1_CACHE_BYTES`. On some architectures this value is determined through the configure-time option `CONFIG_<ARCH>_L1_CACHE_SHIFT`, and on others the value of `L1_CACHE_SHIFT` is hard-coded in the architecture's `include/asm/cache.h` header.

Many open-source projects from Google contain a `base/port.h` header which defines the `CACHELINE_ALIGNED` macro based on an explicit list of architecture detection macros. These header files have often diverged. A token example from the [autofdo](#) project is:

```
// Cache line alignment
#if defined(__i386__) || defined(__x86_64__)
#define CACHELINE_SIZE 64
#elif defined(__powerpc64__)
// TODO(dougkwan) This is the L1 D-cache line size of our Power7 machines.
// Need to check if this is appropriate for other PowerPC64 systems.
#define CACHELINE_SIZE 128
#elif defined(__arm__)
// Cache line sizes for ARM: These values are not strictly correct since
// cache line sizes depend on implementations, not architectures. There
// are even implementations with cache line sizes configurable at boot
// time.
#if defined(__ARM_ARCH_5T__)
#define CACHELINE_SIZE 32
#elif defined(__ARM_ARCH_7A__)
#define CACHELINE_SIZE 64
#endif
#endif

#ifndef CACHELINE_SIZE
// A reasonable default guess. Note that overestimates tend to waste more
// space, while underestimates tend to waste more time.
#define CACHELINE_SIZE 64
#endif

#define CACHELINE_ALIGNED __attribute__((aligned(CACHELINE_SIZE)))
```

Runtime cache-line size

We informatively list a few ways in which the L1 cache-line size can be obtained on different operating systems and architectures at runtime. Libraries such as [hwloc](#) perform these queries, and could also be added to the standard as a separate proposal.

On OSX one would use:

```
sysctlbyname("hw.cachelinesize", &cacheline_size, &sizeof_cacheline_size, 0, 0)
```

On Windows one would use:

```
GetLogicalProcessorInformation(&buf[0], &sizeof_buf);
for (i = 0; i != sizeof_buf / sizeof(SYSTEM_LOGICAL_PROCESSOR_INFORMATION); ++i) {
    if (buf[i].Relationship == RelationCache && buf[i].Cache.Level == 1)
        cacheline_size = buf[i].Cache.LineSize;
```

On Linux one would either use:

```
p = fopen("/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size", "r");
fscanf(p, "%d", &cacheline_size);
```

or:

```
sysconf(_SC_LEVEL1_DCACHE_LINESIZE);
```

On x86 one would use the `CPUID` instruction with `EAX = 80000005h`, which leaves the result in `ECX`, which needs further work to extract.

On ARM one would use `mrs %[ctr], ctr_el0`, which needs further work to extract.